

AI Model Finetuning

Workshop by
Tezan Sahu



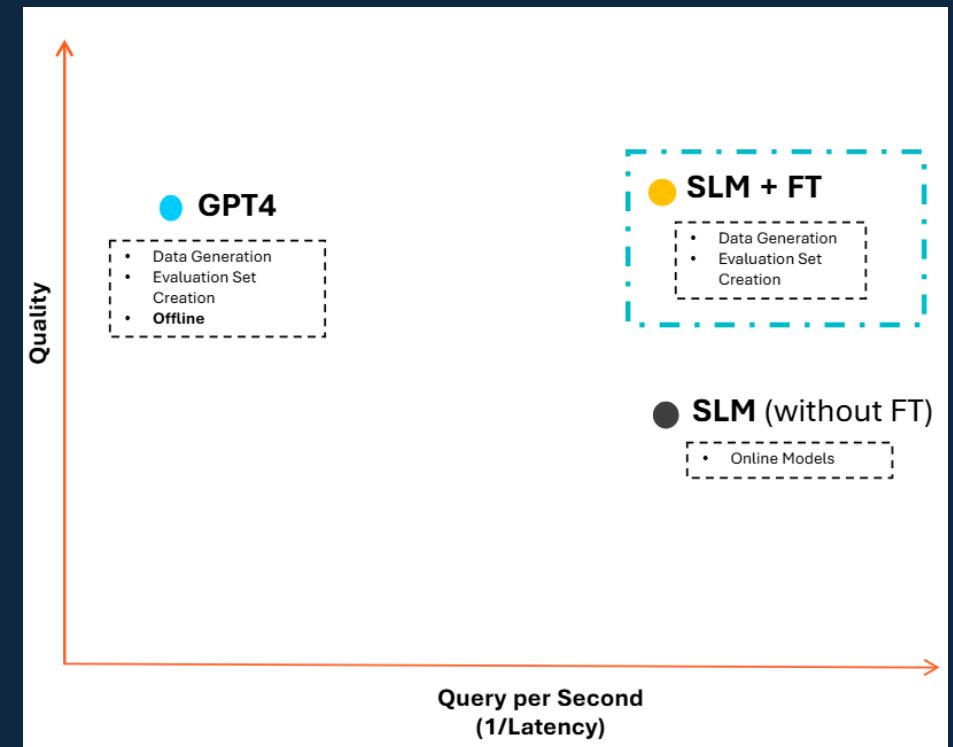
Agenda

- Model Customization
- Demystifying Technical Jargon
- [Live Demo] Fine-tune a Phi 4 Model for Medical Reasoning

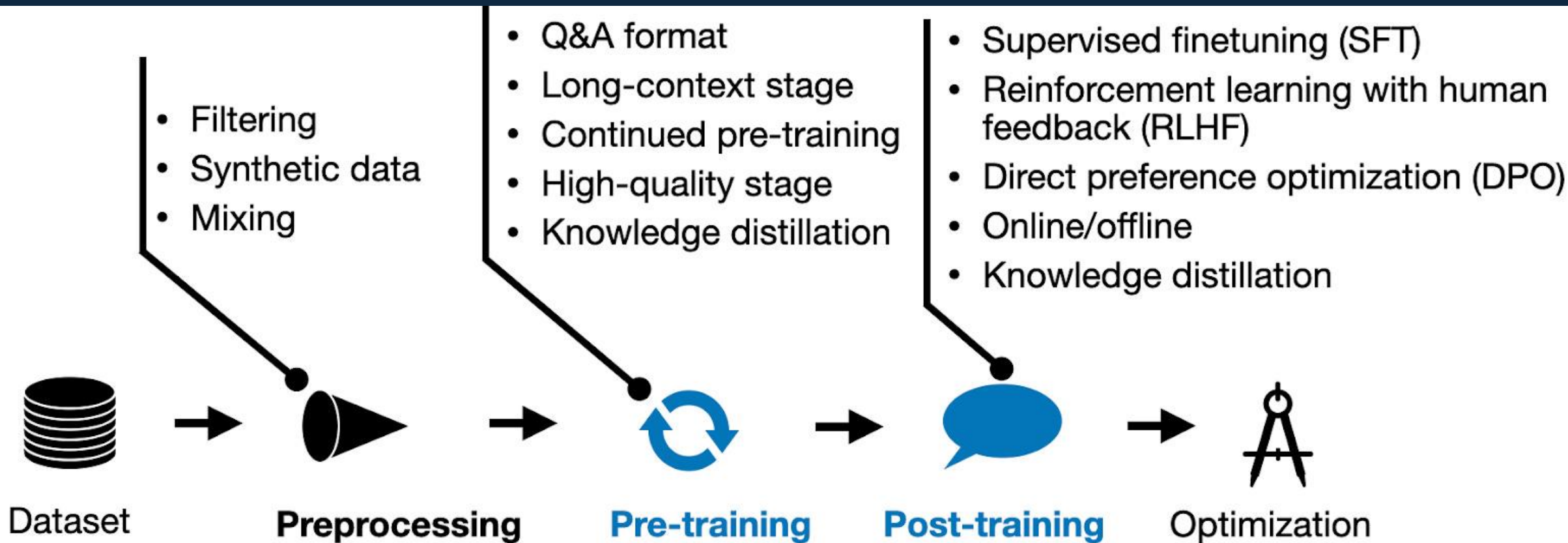
Model Customization

Why Customize AI Models?

- Generic Models vs Specific Domain Tasks
- Reduced Prompt Engineering
- Performance on Edge Devices
- Control & Consistency
- Privacy

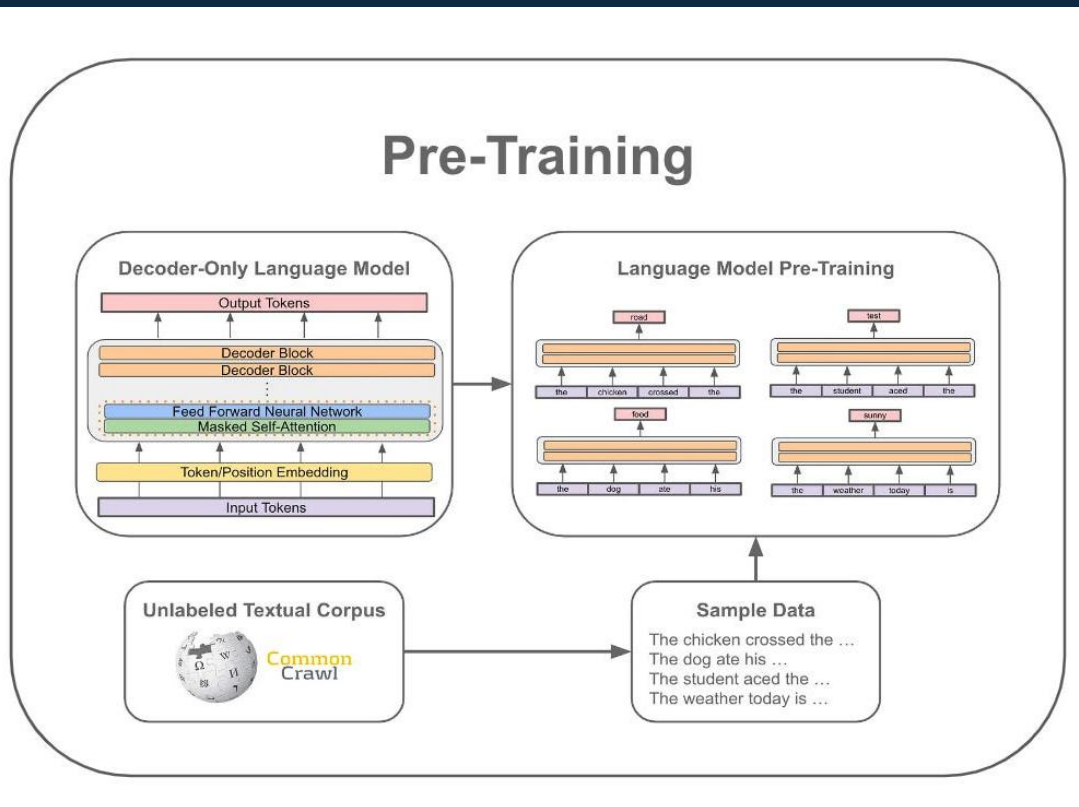


Typical AI Model Training Pipeline



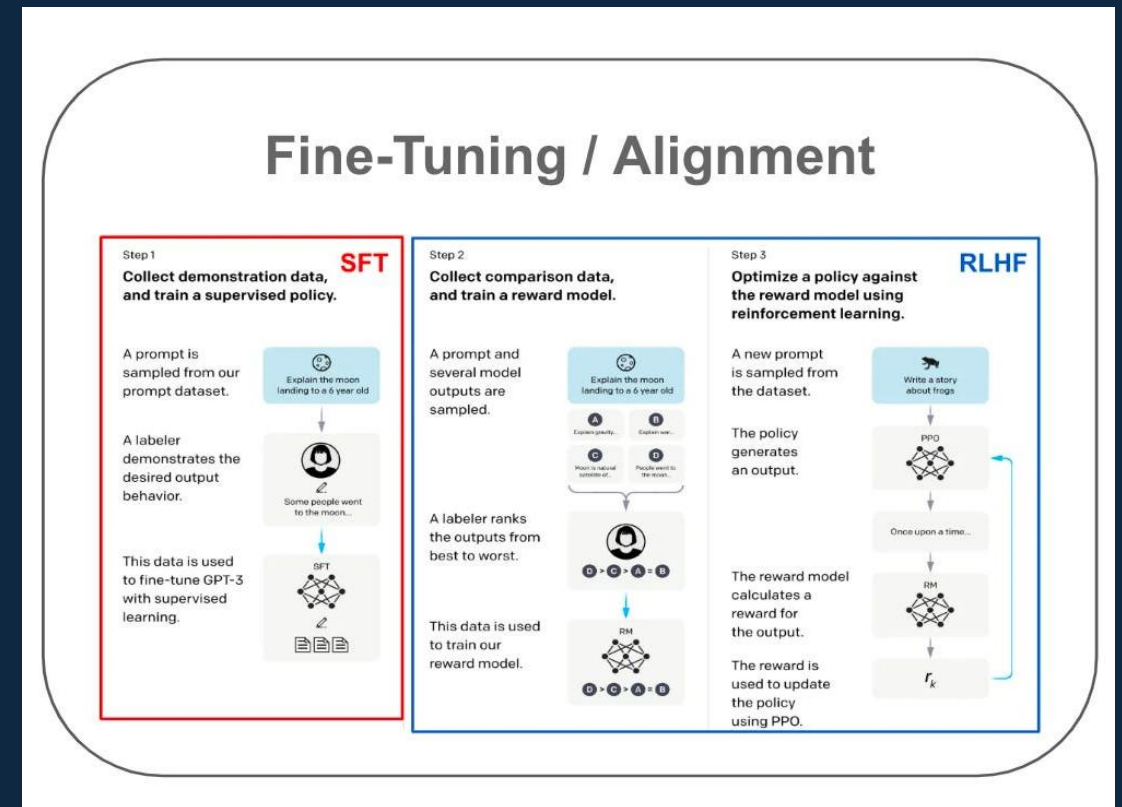
Pre-Training

- Models learn from huge datasets
- Capture grammar, facts, patterns, and language nuances
- Example: Training on Wikipedia, books, articles



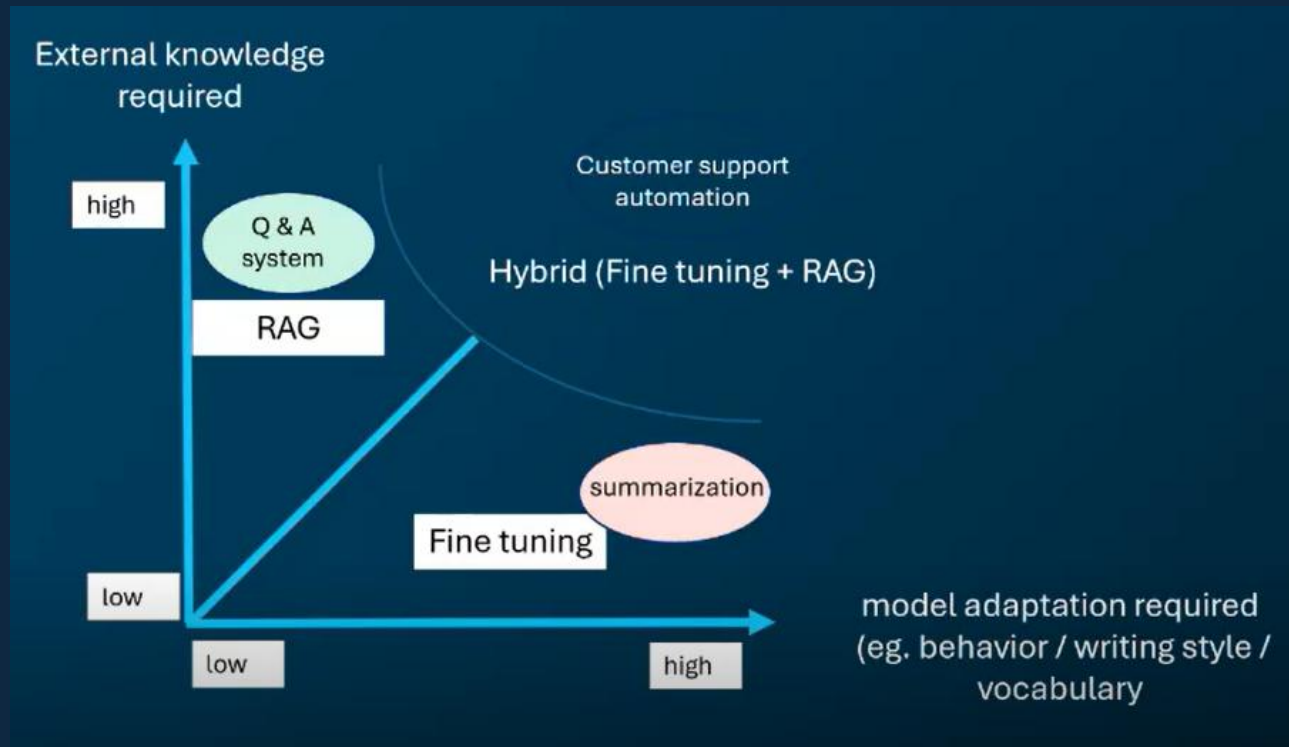
Post-Training

- **Instruction tuning** → teach models how to follow directions using labeled data (SFT)
- **Alignment tuning** → ensure safe and helpful responses (RLHF/DPO)
- Example: Making models friendly, reducing harmful outputs



What is Model Customization?

Once a generic Language Model is out, what do I need to do to make it truly my own?





Demystifying Technical Jargon

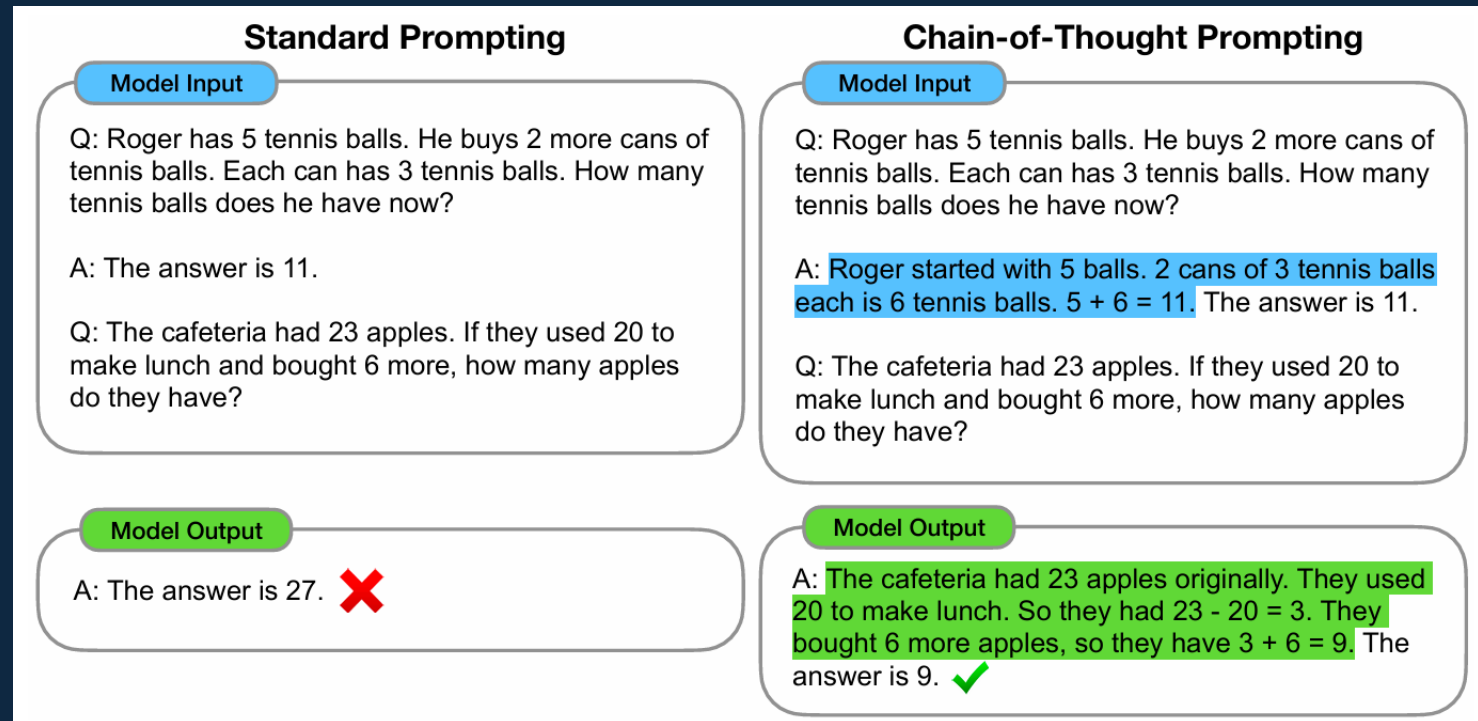
Small Language Models (SLMs)

- Language models with fewer parameters
- **Inference Speed:** Much faster response times
- **Training Cost:** Significantly cheaper to train and fine-tune
- **Use Cases:**
 - Edge deployment (mobile devices, embedded systems)
 - Real-time applications requiring low latency
 - Domain-specific tasks with focused fine-tuning
 - Research and experimentation with limited resources
- **Example:**
 - Llama, Falcon, Vicuna, Alpaca, Phi, Gemma, Mistral, etc.

Characteristic	Small Language Models	Large Language Models
Parameters	<10 billion	10 billion to 1+ trillion
Memory Requirements	Megabytes to a few gigabytes	Tens to hundreds of gigabytes
Inference Speed	Fast	Relatively slower
Energy Consumption	Low	High
Deployment	Can run on edge devices	Typically requires cloud infrastructure
Generalization	Good for specific domains	Better for general knowledge
Context Window	Smaller	Larger
Cost	Lower	Higher

Chain of Thought (CoT)

- A prompting technique that encourages models to show their reasoning process step-by-step, leading to more accurate and explainable outputs.



Reasoning Models

- AI models specifically designed to perform multi-step logical reasoning, mathematical problem-solving, and complex analytical tasks.
- **Training approach:**
 - Trained on datasets with explicit reasoning steps
 - Reinforcement learning to reward correct reasoning
- **Examples:**
 - OpenAI's o1, o3, o4, DeepSeek-R1

The number of distinct real roots of the equation
 $x^7 - 7x - 2 = 0$ is

Thought for 5s

- I'm looking at the polynomial $x^7 - 7x - 2 = 0$. Since its degree is odd (7), it has at least one real root. The derivative $f'(x) = 7(x^3 - 1)(x^3 + 1)$ gives critical points at $x = \pm 1$. Based on the sign of the derivative on different intervals, the function can cross the x-axis up to 3 times, meaning there could be 3 real roots.
- I'm evaluating the polynomial $f(x) = x^7 - 7x - 2$ at various points. At $x = -1$, $f(-1) =$

✓ Done

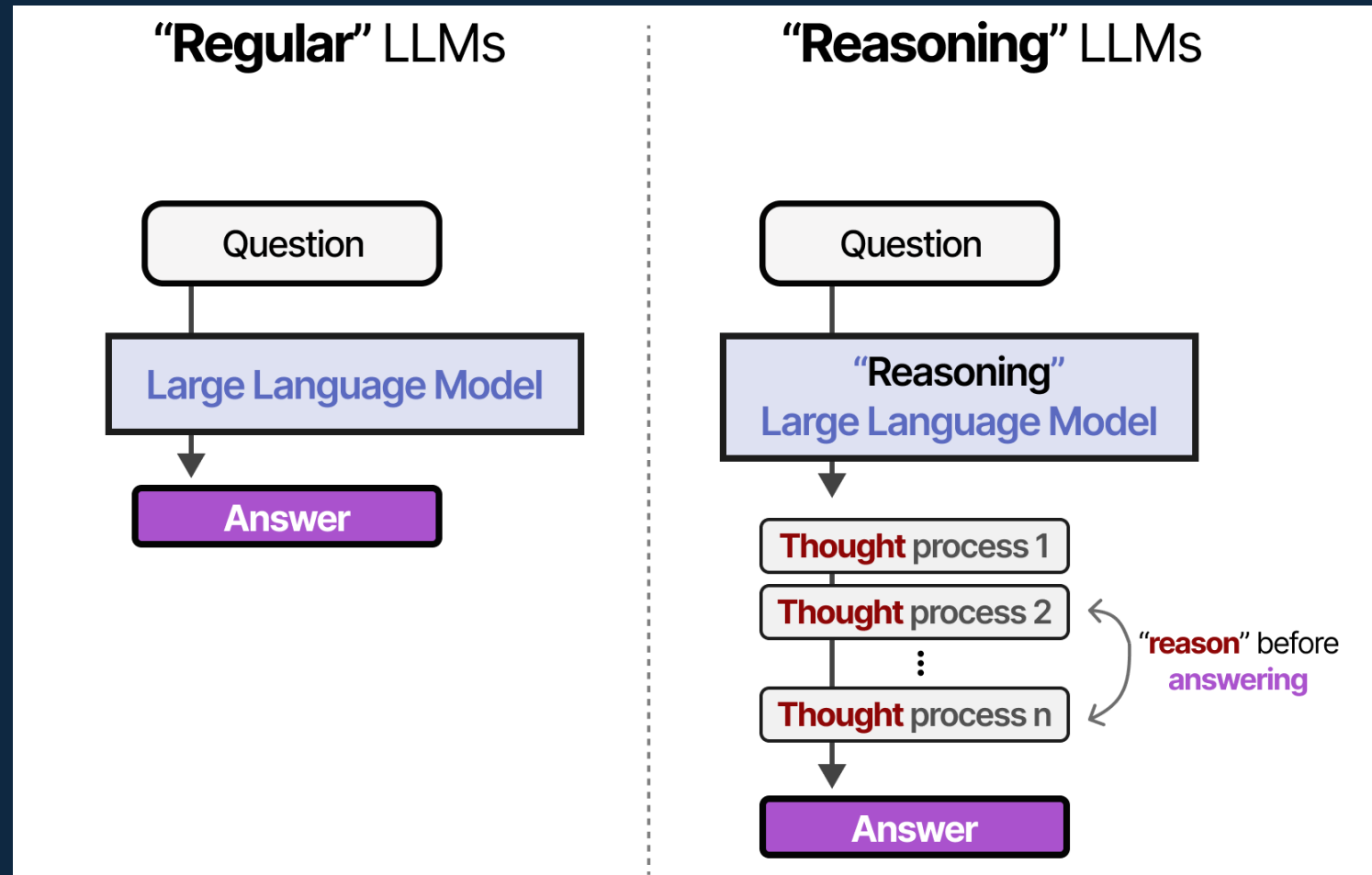
3

Reason: $f(x) = x^7 - 7x - 2$.
 $f'(x) = 7x^6 - 7 = 7(x^6 - 1)$, so $f' > 0$ for $|x| > 1$ and $f' < 0$ for $|x| < 1$. Thus f is increasing on $(-\infty, -1)$, decreasing on $(-1, 1)$, and increasing on $(1, \infty)$.

Evaluate f at ± 1 :
 $f(-1) = -1 + 7 - 2 = 4 > 0$, $f(1) = 1 - 7 - 2 = -8 < 0$.

Since $f(-\infty) = -\infty$ and $f(\infty) = \infty$, by the Intermediate Value Theorem there is exactly one root in each interval $(-\infty, -1)$, $(-1, 1)$, and $(1, \infty)$. Hence three distinct real roots.

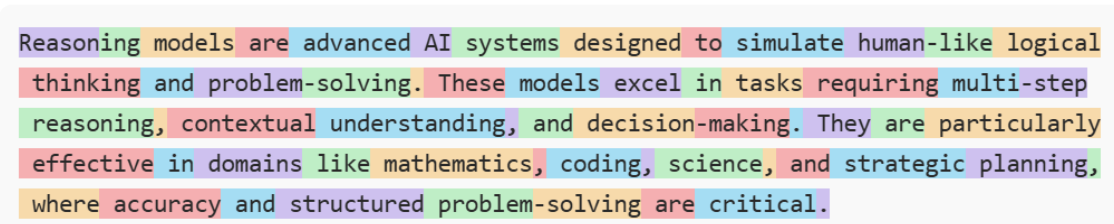
Reasoning Models vs Regular Models



Tokens & Tokenizers

- **Tokens:**

- The basic units that language models process - can be words, parts of words, or even individual characters
- Models process information as tokens, not full sentences
- Different languages tokenize differently

Tokens	Characters
61	379
	

- **Tokenizers:** A tool that splits text into these tokens

- Good tokenization balances vocabulary size and meaning
- Example: Byte Pair Encoding (BPE), SentencePiece, etc.

Chat Template & Special Tokens

- A structured format that organizes conversational data with special tokens to distinguish between different roles and contexts
- It helps the model understand roles and context in multi-turn interactions.

- `<|system|>`: System instructions/context
- `<|user|>`: User messages
- `<|assistant|>`: AI responses
- `<|end|>`: End of message marker
- `<|pad|>`: Padding for batch processing

Training Objective & Loss Function

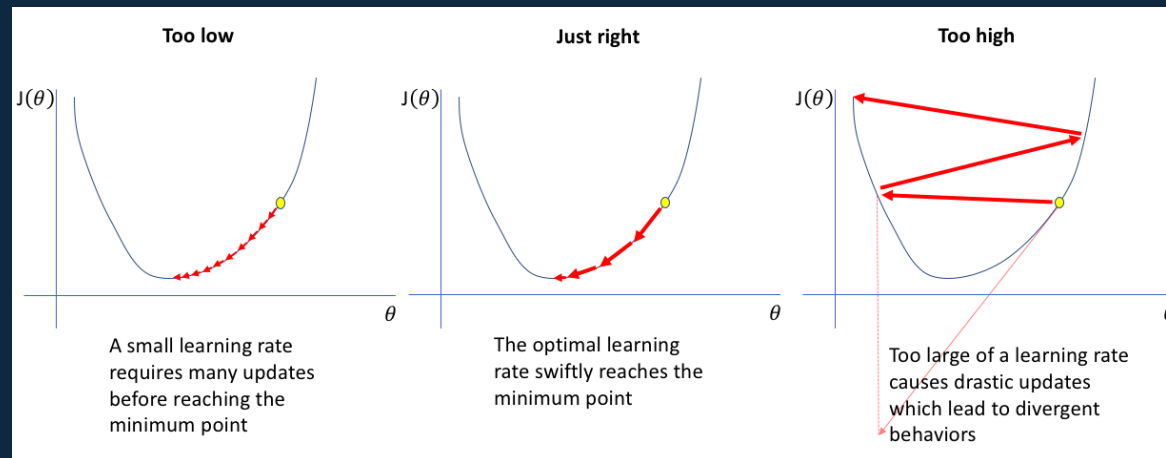
- **Training Objective:** The goal we want the model to achieve during training.
- **Loss Function:** A mathematical way to measure how far the model's predictions are from the desired outputs.
 - **Common Loss Functions:**
 - **Cross-Entropy Loss:** For language modeling - predicts next token
 - **Mean Squared Error:** For regression tasks
 - **Binary Cross-Entropy:** For classification tasks
 - **Custom Loss:** For specific objectives (RLHF, DPO)

Training Hyperparameters: Batch Size

- The number of training examples processed together in one forward/backward pass through the model (before updating model weights)
- **Impact of Batch Size:**
 - **Small Batches (1-32)**
 - More frequent weight updates
 - Higher variance in gradients
 - Less memory usage
 - May help escape local minima
 - **Large Batches (128-1024+)**
 - More stable gradient estimates
 - Better GPU utilization
 - Higher memory requirements
 - May get stuck in suboptimal solutions
- **Sweet Spot: 32-128**

Training Hyperparameters: Learning Rate

- How big steps the model takes when updating its parameters.
 - Critical for training success.
- **Learning Rate Guidelines:**
 - **Too High:** Training unstable, may diverge
 - **Too Low:** Very slow training, may get stuck
 - **Fine-tuning:** Usually $1e-5$ to $5e-4$ (lower than pretraining)



Training Hyperparameters: Others

- **Epochs**

- Number of complete passes through the training dataset.
- Fine-tuning typically uses 1-5 epochs to avoid overfitting.

- **Optimizer**

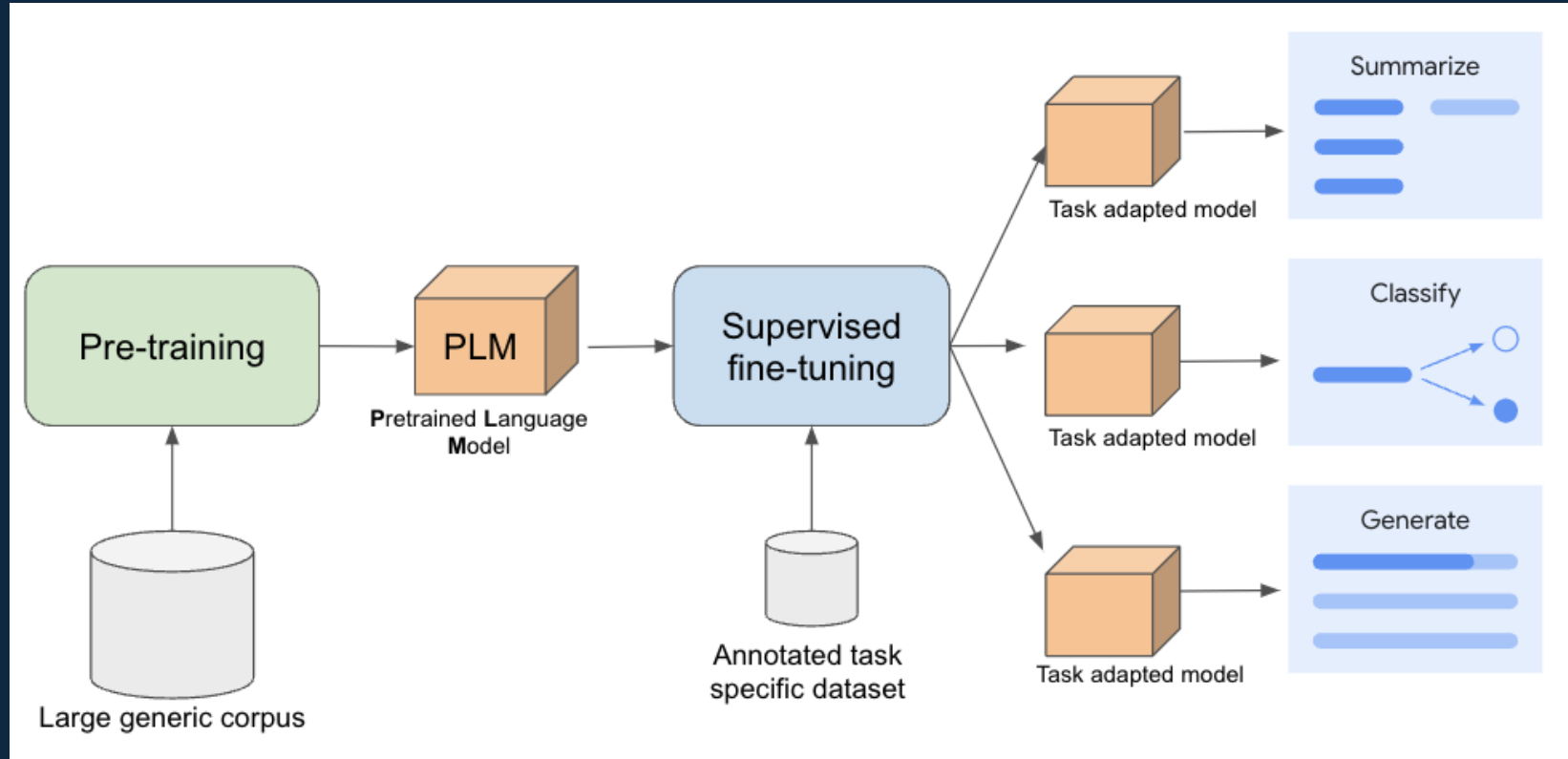
- Algorithm that fine-tunes weights to reduce loss
- **Popular Optimizers:**
 - **AdamW:** Most common for LLMs, handles sparse gradients well
 - **SGD:** Simple but effective for some tasks
 - **AdaFactor:** Memory-efficient alternative to Adam

Quantization

- A technique to reduce model size and memory usage by using lower precision numbers for model parameters
- **Common Quantization Levels:**
 - **FP32 (32-bit float):** Full precision, largest size
 - **FP16 (16-bit float):** Half precision, ~50% size reduction
 - **INT8 (8-bit integer):** ~75% size reduction
 - **INT4 (4-bit integer):** ~87% size reduction
- **Types:**
 - **Post-training Quantization:** Applied after training
 - **Quantization-aware Training:** Model learns to work with lower precision

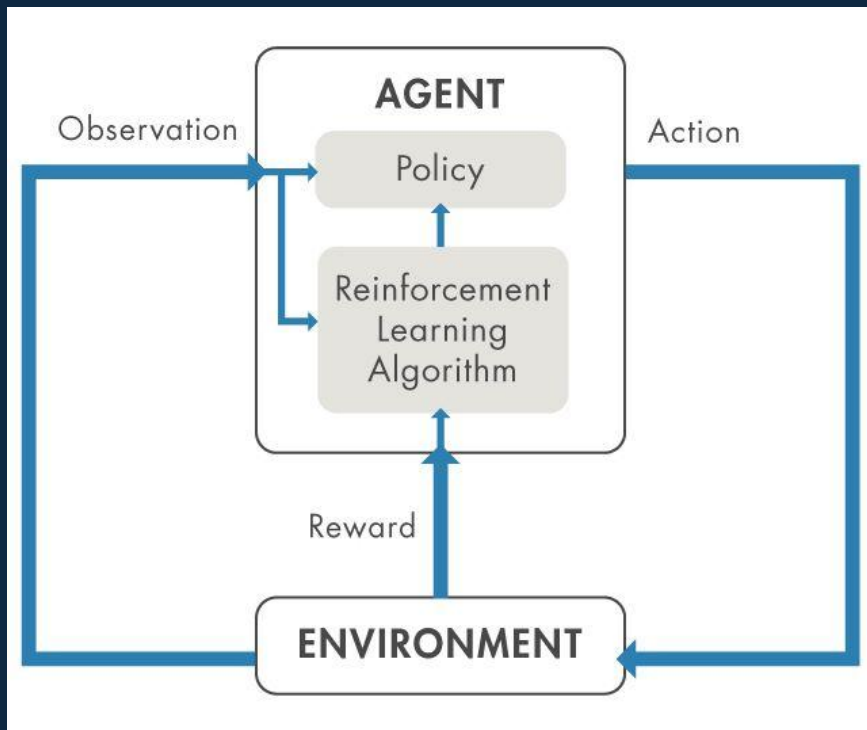
Supervised Fine-tuning (SFT)

- The process of adapting a pre-trained model to a specific task using labeled examples with input-output pairs.



Reinforcement Learning (RL)

- A machine learning approach where an agent learns optimal behavior through interaction with an environment, receiving rewards or penalties for actions.

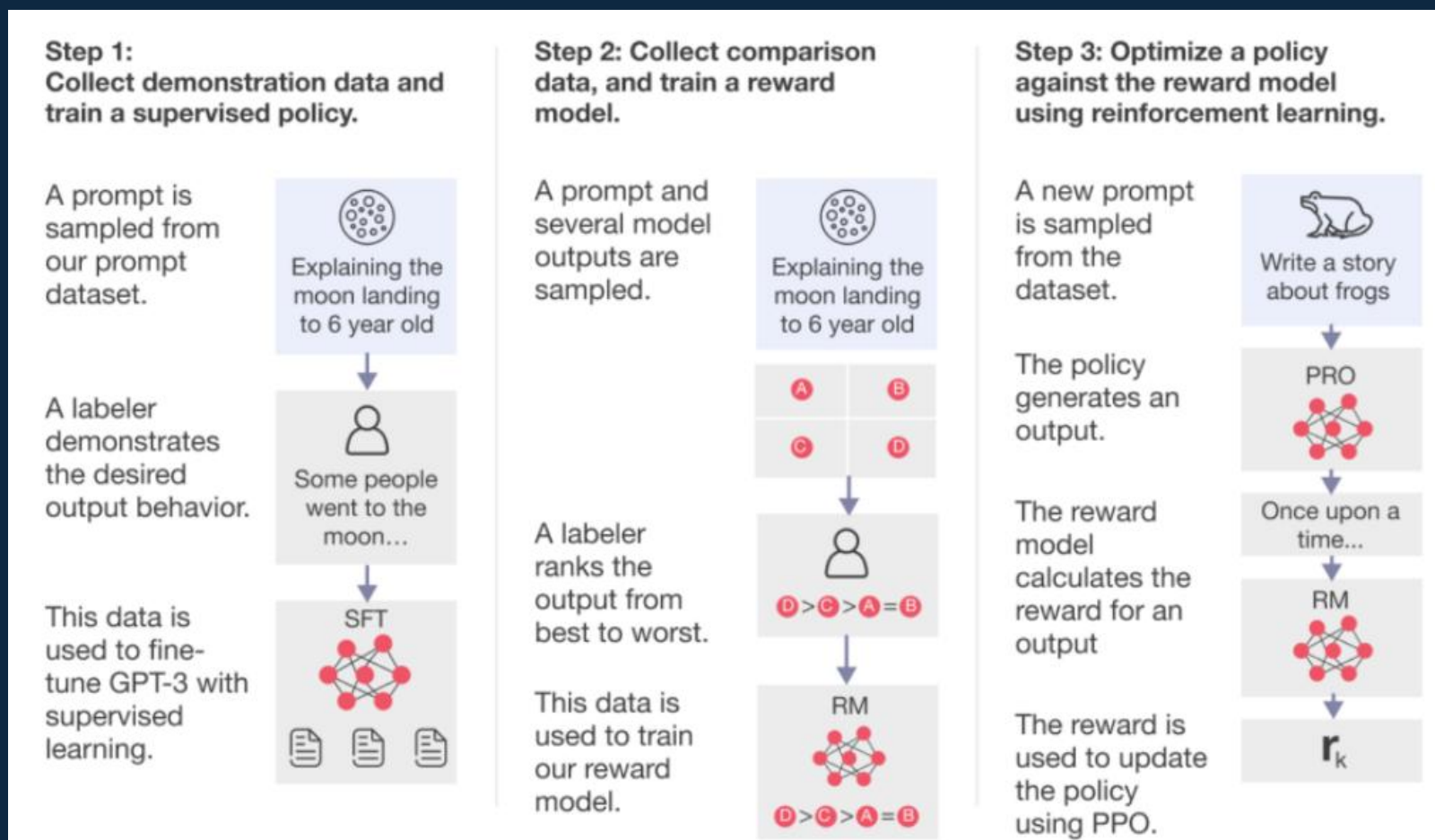


- **Key Components:**

- **Agent:** The language model making decisions
- **Environment:** The context/conversation setting
- **Actions:** Text generation choices
- **Rewards:** Quality scores for generated text
- **Policy:** The model's strategy for generating text

Reinforcement Learning with Human Feedback (RLHF)

- A training method that uses human preferences to guide model behavior

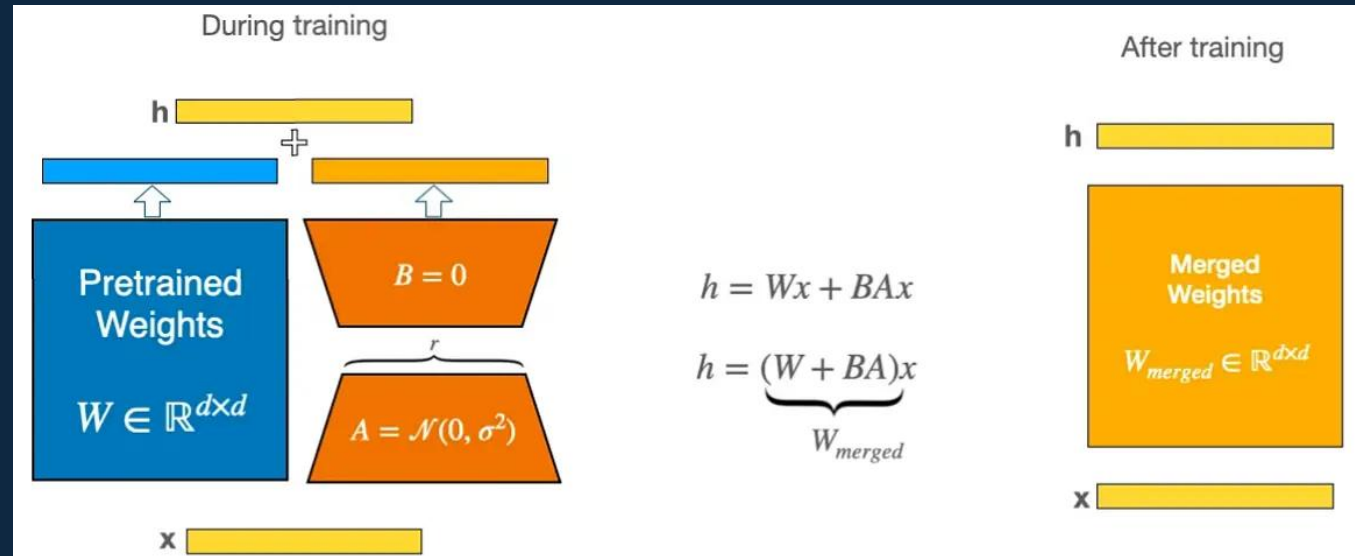


Parameter Efficient Fine-Tuning (PEFT)

- Training methods that update only a small subset of model parameters while keeping most of the original model frozen, dramatically reducing computational requirements.
- **Common PEFT Methods:**
 - LoRA: Low-rank adaptation (most popular)
 - Adapters: Small neural network modules
 - Prefix Tuning: Optimize input prefixes
- **Typical Results:**
 - Train 0.1-1% of parameters
 - Achieve 90-95% of full fine-tuning performance
 - Reduce memory usage by 70-90%
 - Enable fine-tuning on consumer GPU

Low Rank Adaptation (LoRA)

- A PEFT technique that decomposes weight updates into low-rank matrices, allowing efficient fine-tuning with minimal parameter overhead.



Example:

Original matrix: $4096 \times 4096 = 16\text{M}$ parameters

LoRA with rank 64: $(4096 \times 64) + (64 \times 4096) = 524\text{K}$ parameters

Reduction: 97% fewer parameters!

Time for some Live Action!



Quiz Time!!!



Scan the QR Code
Use the code 4742 9780 on menti.com

That's All Folks!

For More Interesting Stuff on AI , Data Science & the Latest Technologies...

